

Проектное задание по машинному обучению

«Разработка и исследование ML-модели для решения прикладной задачи»

Делали: Ченцов Егор, Калмыков Амирхан

Группа: ИД24-3

Цель

Разработать и обосновать модель машинного обучения для решения реальной прикладной задачи в любой предметной области с полным циклом: данные → модель → оценка → интерпретация → сохранение модели.

Область

- Бизнес / маркетинг (отток клиентов, сегментация, рекомендации)

Этапы проекта

3.1 Постановка задачи

Бизнес-задача:

Разработка системы прогнозирования оттока клиентов для телекоммуникационной компании

В условиях высокой конкуренции на рынке телекоммуникационных услуг удержание существующих клиентов становится более приоритетной задачей, чем привлечение новых. Данный проект направлен на **создание ML-модели**, способной выявлять клиентов с высоким риском расторжения контракта на ранней стадии.

Бизнес-цель Минимизировать финансовые потери (Churn Loss) за счет своевременного предложения персонализированных условий. Мы решаем задачу бинарной классификации, где целевая переменная — факт ухода клиента в текущем квартале.

3.2 Данные

Используется набор данных **Telco Customer Churn** от IBM.

- 7043 наблюдения, 33 признака.
- Целевая переменная: `Churn Value` (или `Churn Label`).
- Признаки: демографические (пол, возраст, наличие партнёра, иждивенцев), поведенческие (длительность обслуживания, услуги), договорные (тип

контракта, способ оплаты), финансовые (ежемесячные и общие платежи).

3.3 Предобработка

установка библиотеки

```
In [1]: !pip install kagglehub
```

```
Requirement already satisfied: kagglehub in c:\users\home\anaconda3\lib\site-pack
ages (1.0.1)
Requirement already satisfied: kagglesdk<1.0,>=0.1.22 in c:\users\home\anaconda3
\lib\site-packages (from kagglehub) (0.1.23)
Requirement already satisfied: packaging in c:\users\home\anaconda3\lib\site-pack
ages (from kagglehub) (23.2)
Requirement already satisfied: pyyaml in c:\users\home\anaconda3\lib\site-package
s (from kagglehub) (6.0.1)
Requirement already satisfied: requests in c:\users\home\anaconda3\lib\site-packa
ges (from kagglehub) (2.32.2)
Requirement already satisfied: tqdm in c:\users\home\anaconda3\lib\site-packages
 (from kagglehub) (4.66.4)
Requirement already satisfied: protobuf in c:\users\home\anaconda3\lib\site-packa
ges (from kagglesdk<1.0,>=0.1.22->kagglehub) (6.33.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\home\anaconda
3\lib\site-packages (from requests->kagglehub) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\home\anaconda3\lib\site-p
ackages (from requests->kagglehub) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\home\anaconda3\lib
\site-packages (from requests->kagglehub) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\home\anaconda3\lib
\site-packages (from requests->kagglehub) (2024.7.4)
Requirement already satisfied: colorama in c:\users\home\anaconda3\lib\site-packa
ges (from tqdm->kagglehub) (0.4.6)
```

используемые библиотеки

```
In [156... import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import scipy.stats as stats
import statsmodels.api as sm

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
import joblib

%matplotlib inline
sns.set_style('whitegrid')
```

Источники датасета

<https://community.ibm.com/community/user/blogs/steven-macko/2019/07/11/telco->

customer-churn-1113

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

In [7]: `import kagglehub`

```
path = kagglehub.dataset_download("yeanzc/telco-customer-churn-ibm-dataset")
print("Путь:", path)
```

Путь: C:\Users\Home\.cache\kagglehub\datasets\yeanzc\telco-customer-churn-ibm-dataset\versions\1

In [9]: `#df = pd.read_csv(f"{path}/WA_Fn-UseC_-Telco-Customer-Churn.csv")`
`db = pd.read_excel(f"{path}/Telco_customer_churn.xlsx")`
`db_raw = db.copy()`

In [10]: `print("Размер данных:", db.shape)`

Размер данных: (7043, 33)

In [17]: `db.head()`

Out[17]:

	CustomerID	Count	Country	State	City	Zip Code	Lat Long	Latitude	Lo
0	3668-QPYBK	1	United States	California	Los Angeles	90003	33.964131, -118.272783	33.964131	-118
1	9237-HQITU	1	United States	California	Los Angeles	90005	34.059281, -118.30742	34.059281	-118
2	9305-CDSKC	1	United States	California	Los Angeles	90006	34.048013, -118.293953	34.048013	-118
3	7892-POOKP	1	United States	California	Los Angeles	90010	34.062125, -118.315709	34.062125	-118
4	0280-XJGEX	1	United States	California	Los Angeles	90015	34.039224, -118.266293	34.039224	-118

5 rows × 33 columns



In [19]: `db.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            7043 non-null   object
1   Count                  7043 non-null   int64
2   Country                7043 non-null   object
3   State                  7043 non-null   object
4   City                   7043 non-null   object
5   Zip Code               7043 non-null   int64
6   Lat Long               7043 non-null   object
7   Latitude               7043 non-null   float64
8   Longitude              7043 non-null   float64
9   Gender                 7043 non-null   object
10  Senior Citizen         7043 non-null   object
11  Partner                7043 non-null   object
12  Dependents             7043 non-null   object
13  Tenure Months          7043 non-null   int64
14  Phone Service          7043 non-null   object
15  Multiple Lines         7043 non-null   object
16  Internet Service       7043 non-null   object
17  Online Security        7043 non-null   object
18  Online Backup          7043 non-null   object
19  Device Protection      7043 non-null   object
20  Tech Support           7043 non-null   object
21  Streaming TV           7043 non-null   object
22  Streaming Movies       7043 non-null   object
23  Contract               7043 non-null   object
24  Paperless Billing       7043 non-null   object
25  Payment Method         7043 non-null   object
26  Monthly Charges        7043 non-null   float64
27  Total Charges          7043 non-null   object
28  Churn Label            7043 non-null   object
29  Churn Value            7043 non-null   int64
30  Churn Score            7043 non-null   int64
31  CLTV                   7043 non-null   int64
32  Churn Reason           1869 non-null   object
dtypes: float64(3), int64(6), object(24)
memory usage: 1.8+ MB

```

Преобразуем Total Charges в число, пропуски заполним медианой

```

In [22]: db['Total Charges'] = pd.to_numeric(db['Total Charges'], errors='coerce')
db['Total Charges'] = db['Total Charges'].fillna(db['Total Charges'].median())

```

Создаем новый признак: количество подключенных дополнительных услуг

```

In [25]: services = ['Phone Service', 'Multiple Lines', 'Internet Service',
                    'Online Security', 'Online Backup', 'Device Protection',
                    'Tech Support', 'Streaming TV', 'Streaming Movies']

```

Считаем количество услуг и отношение ежемесячного платежа к общему

```

In [28]: db['Total Services'] = db[services].apply(lambda x: x.str.contains('Yes').sum(),
db['Charges Ratio'] = db['Monthly Charges'] / (db['Total Charges'] + 1)

```

Для повышения предсказательной способности модели был введен признак Charges Ratio (отношение текущего платежа к общему), который служит индикатором интенсивности потребления услуг относительно срока лояльности. Также был создан агрегированный признак Total Services, отражающий степень вовлеченности клиента в экосистему компании (количество подключенных дополнительных опций)

Удалили неинформативные столбцы

```
In [32]: drop_cols = ['CustomerID', 'Count', 'Country', 'State', 'City', 'Zip Code',  
                    'Lat Long', 'Latitude', 'Longitude', 'Churn Reason']  
db.drop(columns=drop_cols, inplace=True)
```

Кодирование категориальных признаков

```
In [35]: categorical_cols = db.select_dtypes(include=['object']).columns  
for col in categorical_cols:  
    db[col] = db[col].astype('category').cat.codes
```

Масштабирование числовых признаков (кроме целевой)

```
In [38]: num_cols = db.select_dtypes(include=['int64', 'float64']).columns.drop('Churn Value')  
scaler = StandardScaler()  
db[num_cols] = scaler.fit_transform(db[num_cols])
```

```
In [40]: print("Размер данных после предобработки:", db.shape)
```

Размер данных после предобработки: (7043, 25)

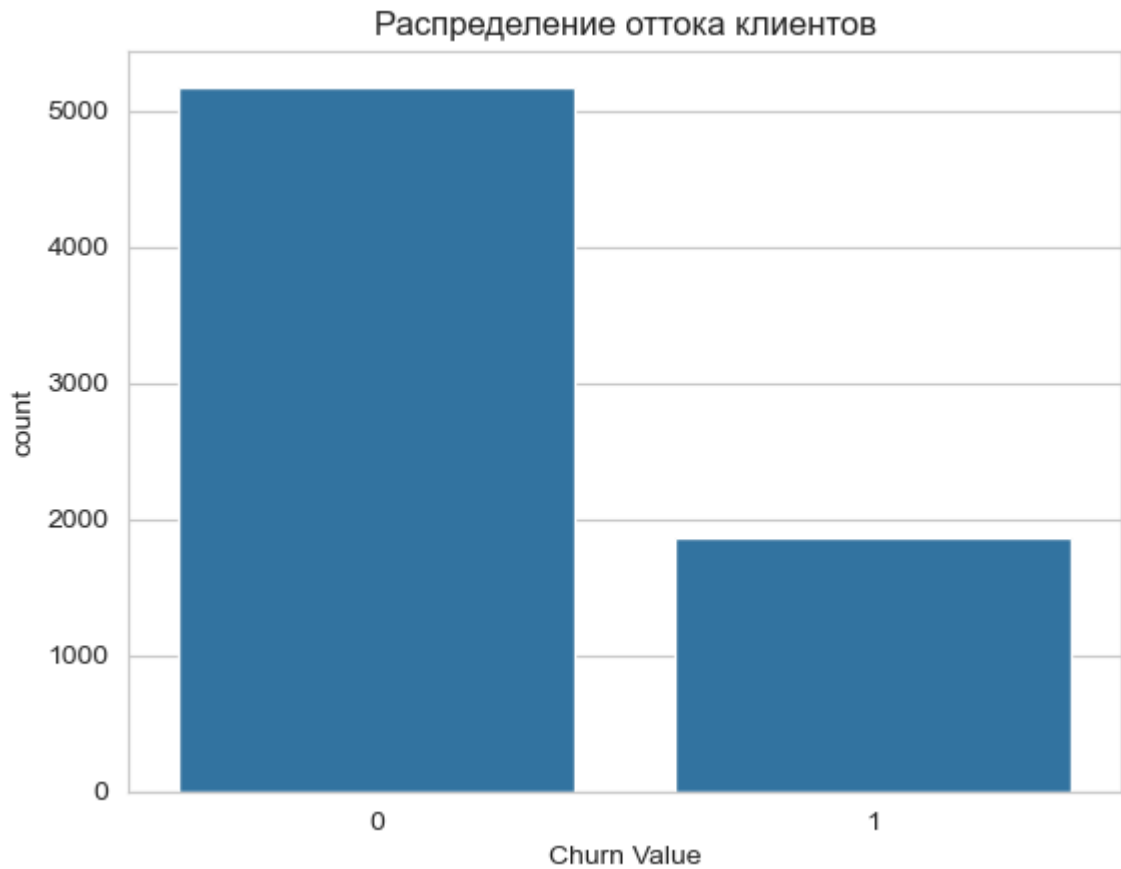
```
In [42]: print(f"Всего клиентов: {len(db)}")  
print(f"Ушедших: {(db['Churn Value'] == 1).sum()} ({(db['Churn Value'] == 1).mean()*100}%)")  
print(f"Оставшихся: {(db['Churn Value'] == 0).sum()} ({(db['Churn Value'] == 0).mean()*100}%)")
```

Всего клиентов: 7043
Ушедших: 1869 (26.5%)
Оставшихся: 5174 (73.5%)

3.4 EDA

Распределение целевой переменной

```
In [46]: sns.countplot(data=db, x='Churn Value')  
plt.title('Распределение оттока клиентов')  
plt.show()
```



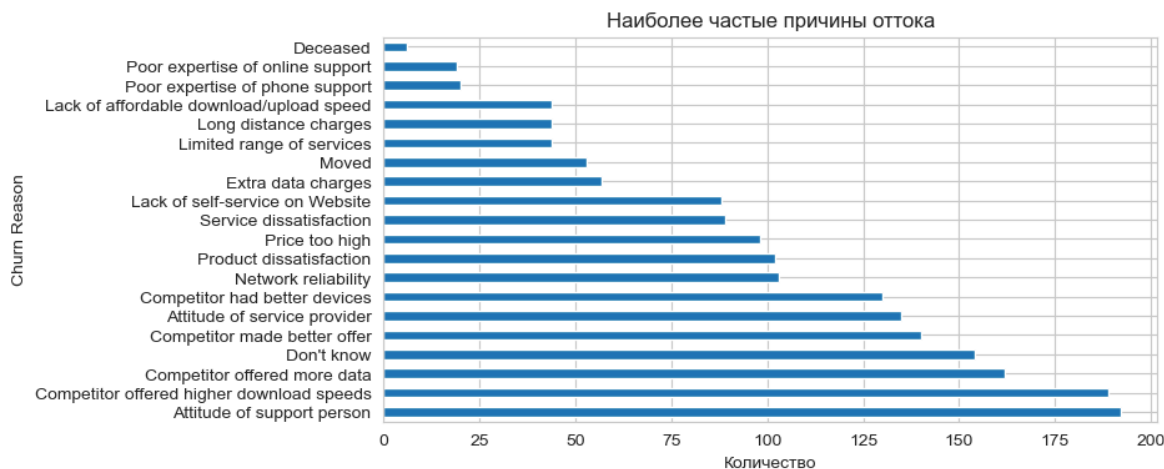
```
In [48]: print(db['Churn Value'].value_counts(normalize=True))
```

```
Churn Value
0    0.73463
1    0.26537
Name: proportion, dtype: float64
```

Вывод: Доля оттока составляет около 26.5% – классы несбалансированы, но не критично

Причины оттока

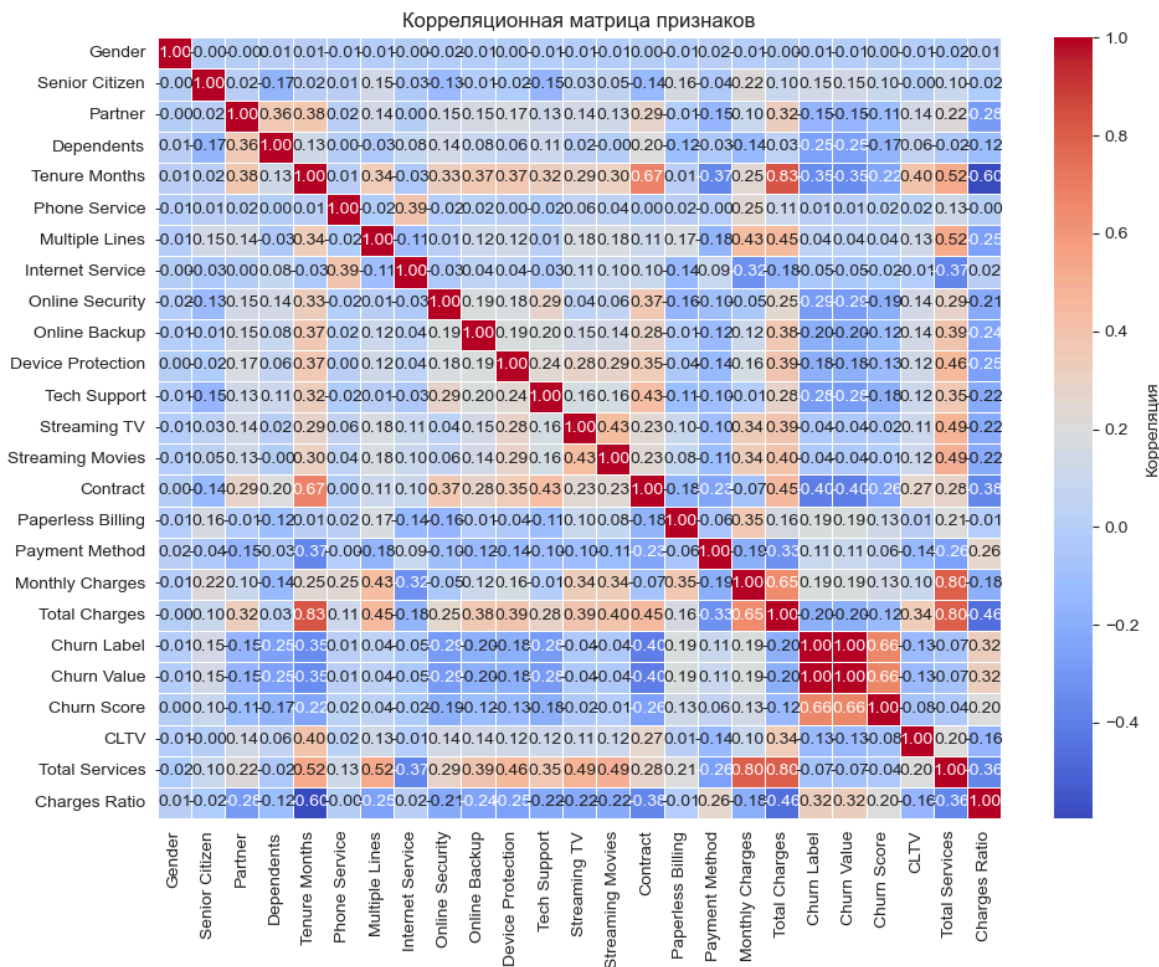
```
In [52]: churn_reason_counts = db_raw['Churn Reason'].dropna().value_counts()
plt.figure(figsize=(8,4))
churn_reason_counts.plot(kind='barh')
plt.title('Наиболее частые причины оттока')
plt.xlabel('Количество')
plt.show()
```



Вывод: Главные причины – отношение сотрудника поддержки, предложения конкурентов, скорость интернета, недостаток данных, цены

Карта корреляций

```
In [56]: plt.figure(figsize=(10, 8))
corr_matrix = db.corr(numeric_only=True)
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5,
plt.title('Корреляционная матрица признаков')
plt.tight_layout()
plt.show()
```

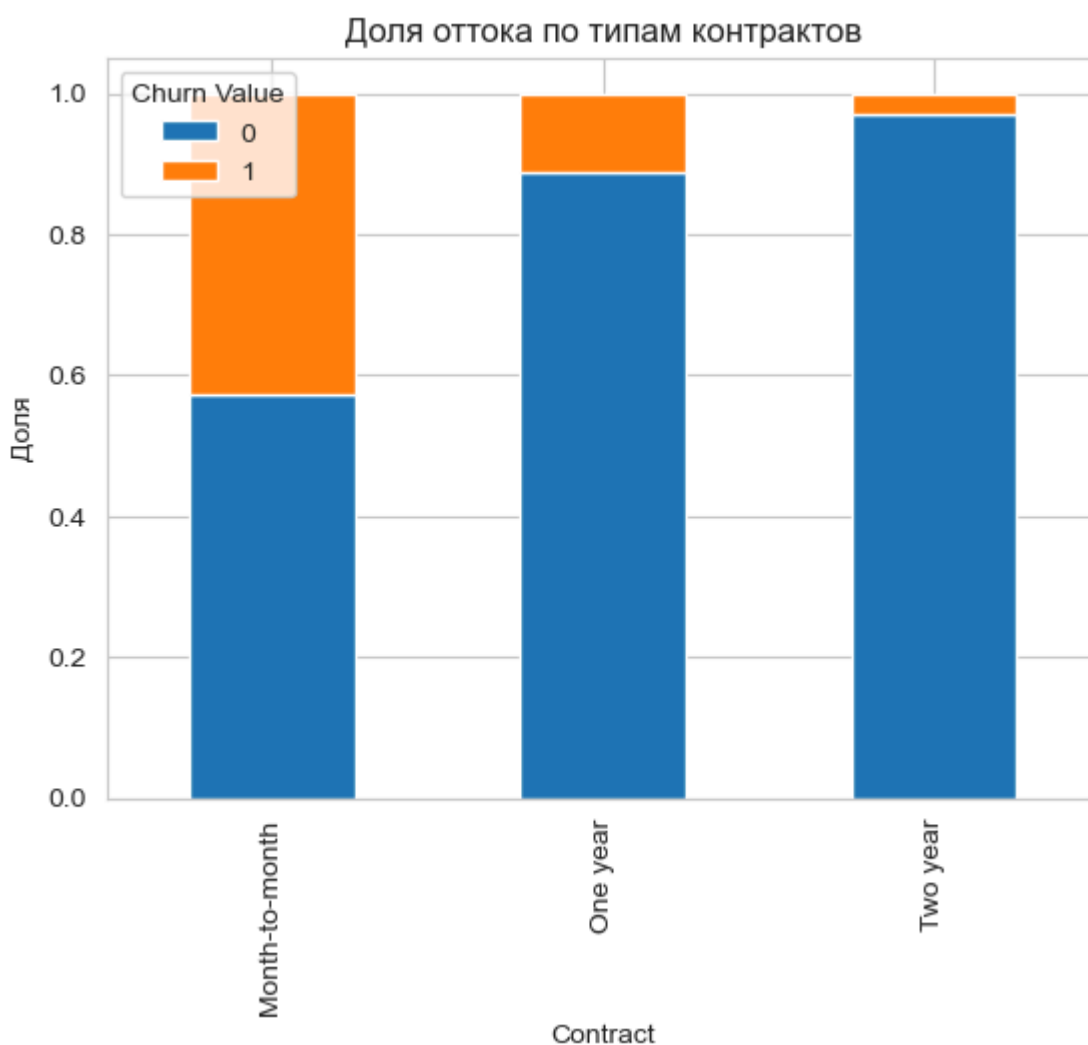


Выводы:

- Tenure Months сильно коррелирует с Total Charges (0.83) и Contract (0.67).
- Churn Value отрицательно коррелирует с Tenure Months (-0.35) и Contract (-0.4).

Зависимость оттока от типа контракта

```
In [60]: contract_churn = pd.crosstab(db_raw['Contract'], db_raw['Churn Value'], normalize=True)
contract_churn.plot(kind='bar', stacked=True)
plt.title('Доля оттока по типам контрактов')
plt.ylabel('Доля')
plt.show()
```



Вывод: У клиентов с ежемесячным контрактом отток $\approx 45\%$, у годовых – $\approx 11\%$, у двухлетних – $\approx 3\%$. Долгосрочные контракты значительно снижают отток

Анализ tenure для ушедших клиентов

```
In [64]: churned = db_raw[db_raw['Churn Value'] == 1]
sns.histplot(churned['Tenure Months'], bins=30, kde=True)
plt.title('Распределение длительности обслуживания среди ушедших клиентов')
plt.show()
```



```
In [66]: print("Средний tenure ушедших:", churned['Tenure Months'].mean())
print("Наиболее частый tenure ушедших:", churned['Tenure Months'].mode()[0])
```

Средний tenure ушедших: 17.979133226324237

Наиболее частый tenure ушедших: 1

Вывод: Многие клиенты уходят в первый месяц, средний tenure ушедших ≈ 18 месяцев, что говорит о потерях ценных клиентов после определённого периода

```
In [69]: print(f"Класс несбалансирован: {(db['Churn Value'].value_counts()[1] / len(db) * 100):.1f}%")
print(f"Наибольшая корреляция с оттоком: {corr_matrix['Churn Value'].drop('Churn Value').corr().max():.3f}")
```

Класс несбалансирован: 26.5% оттока (класс 1)

Наибольшая корреляция с оттоком: {'Churn Label': 1.0, 'Churn Score': 0.6648970311816242, 'Contract': 0.396712629209845}

Анализ распределения признаков показал, что наиболее критическим периодом для удержания является первый месяц обслуживания (High Early Churn). При этом выявлена статистически значимая связь между типом контракта и вероятностью ухода: клиенты с помесечной оплатой склонны к оттоку в 15 раз чаще, чем клиенты с двухлетним контрактом. Это подтверждает необходимость фокусировки модели на поведении новых пользователей

3.5 Моделирование

Список столбцов, которые нужно убрать

```
In [74]: leakage_cols = ['Churn Value', 'Churn Label', 'Churn Score', 'CLTV']
```

```
In [76]: X = db.drop(columns=leakage_cols, errors='ignore')
y = db['Churn Value']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```
X = db.drop(['Churn Value'], axis=1) y = db['Churn Value'] X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

```
In [78]: print(f"train: {X_train.shape}")
print(f"test: {X_test.shape}")
```

```
train: (5634, 21)
test: (1409, 21)
```

Логистическая регрессия

```
In [81]: lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
```

```
Out[81]: ▼ LogisticRegression ⓘ ?
          ► Parameters
```

```
In [83]: y_pred_lr = lr.predict(X_test)
y_proba_lr = lr.predict_proba(X_test)[: , 1]
```

Дерево решений

```
In [86]: dt = DecisionTreeClassifier(random_state=42, max_depth=20)
dt.fit(X_train, y_train)
```

```
Out[86]: ▼ DecisionTreeClassifier ⓘ ?
          ► Parameters
```

```
In [88]: y_pred_dt = dt.predict(X_test)
y_proba_dt = dt.predict_proba(X_test)[: , 1]
```

Случайный лес

```
In [91]: rf = RandomForestClassifier(random_state=42, n_jobs=-1, n_estimators=100)
rf.fit(X_train, y_train)
```

```
Out[91]: ▼ RandomForestClassifier ⓘ ?
          ► Parameters
```

```
In [93]: y_pred_rf = rf.predict(X_test)
y_proba_rf = rf.predict_proba(X_test)[: , 1]
```

Градиентный бустинг

```
In [96]: gb = GradientBoostingClassifier(random_state=42, n_estimators=100)
gb.fit(X_train, y_train)
```

```
Out[96]: ▾ GradientBoostingClassifier ⓘ ⓘ
          ▶ Parameters
```

```
In [97]: y_pred_gb = gb.predict(X_test)
y_proba_gb = gb.predict_proba(X_test)[:, 1]
```

3.6 Оценка качества

```
In [101... models = ['Logistic Regression', 'Decision Tree', 'Random Forest', 'Gradient Boosting']
preds = [y_pred_lr, y_pred_dt, y_pred_rf, y_pred_gb]
probas = [y_proba_lr, y_proba_dt, y_proba_rf, y_proba_gb]
```

```
In [103... results = []
for name, pred, proba in zip(models, preds, probas):
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
        'Precision': precision_score(y_test, pred),
        'Recall': recall_score(y_test, pred),
        'F1': f1_score(y_test, pred),
        'ROC-AUC': roc_auc_score(y_test, proba)
    })
```

```
In [105... results_df = pd.DataFrame(results)
results_df
```

```
Out[105...
      Model Accuracy Precision Recall    F1 ROC-AUC
0 Logistic Regression  0.811214  0.665644  0.580214  0.620000  0.854998
1   Decision Tree    0.726757  0.486553  0.532086  0.508301  0.668237
2   Random Forest    0.787083  0.624161  0.497326  0.553571  0.833991
3 Gradient Boosting    0.814053  0.686667  0.550802  0.611276  0.854043
```

- **Градиентный бустинг (Gradient Boosting) и Логистическая регрессия** показали лучшие результаты по Accuracy (0.81) и ROC-AUC (0.85).
- **Recall:** У всех моделей Recall (полнота) заметно ниже Precision. Это значит, что модель хорошо определяет тех, кто точно уйдет, но пропускает часть клиентов, которые на самом деле планируют расторгнуть контракт. **Модель осторожничает**
- Для внедрения рекомендуем использовать **Gradient Boosting**, так как он показал наиболее сбалансированные метрики

Модели показывают стабильные результаты без признаков переобучения. Значение ROC-AUC 0.85 говорит о высоком качестве разделения классов

In [158...

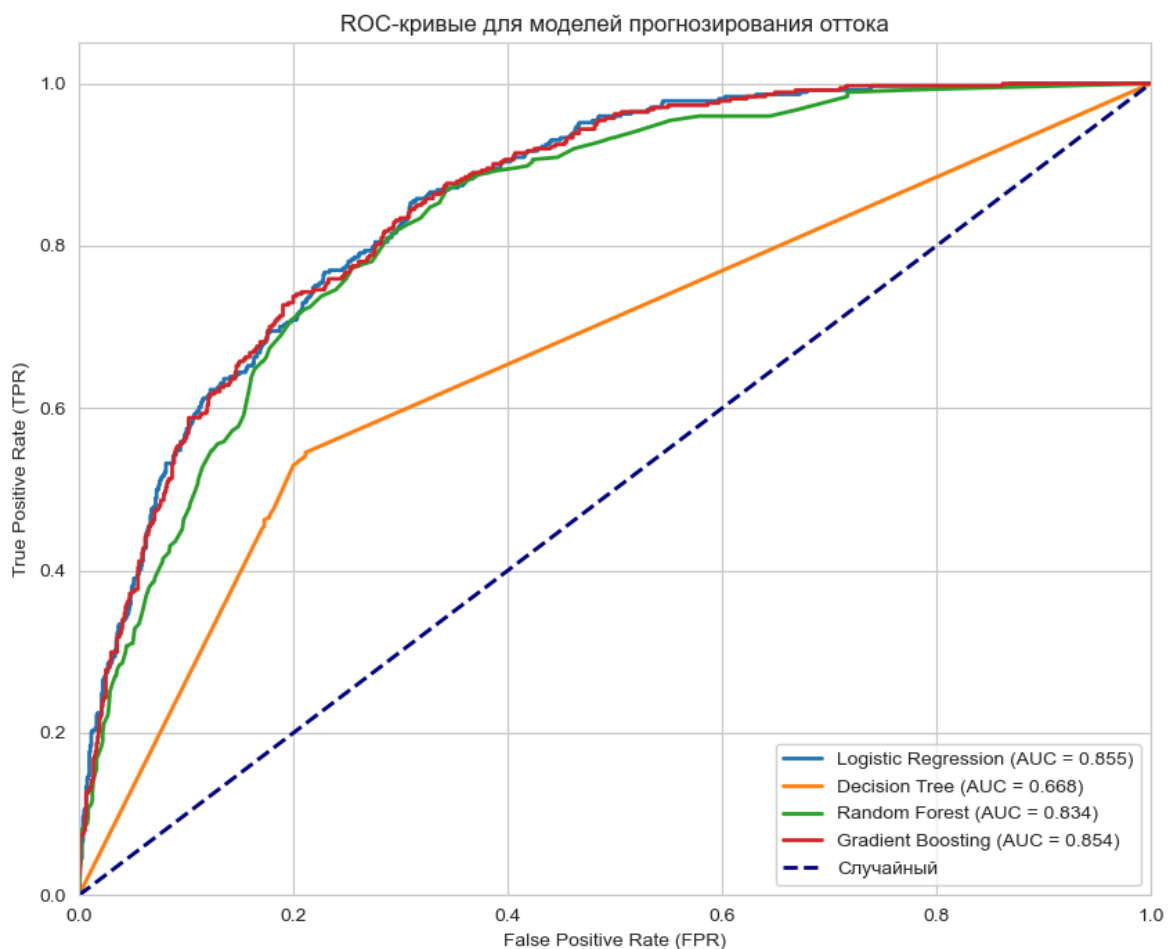
```
plt.figure(figsize=(10, 8))

models_roc = [
    ('Logistic Regression', y_proba_lr),
    ('Decision Tree', y_proba_dt),
    ('Random Forest', y_proba_rf),
    ('Gradient Boosting', y_proba_gb)
]

for name, proba in models_roc:
    fpr, tpr, _ = roc_curve(y_test, proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC = {roc_auc:.3f})')

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Случайный')

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC-кривые для моделей прогнозирования оттока')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



3.7 Подбор гиперпараметров

In [109...

```
param_grid = {
    'n_estimators': [100, 200, 300],
```

```

    'max_depth': [10, 15, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

```

```

In [111...] rf_tune = RandomForestClassifier(random_state=42, n_jobs=-1)
            grid = GridSearchCV(rf_tune, param_grid, cv=5, scoring='roc_auc', n_jobs=-1, ver
            grid.fit(X_train, y_train)

```

Fitting 5 folds for each of 108 candidates, totalling 540 fits

```

Out[111...]
  ▸ GridSearchCV ⓘ ?
  ▸ best_estimator_: RandomForestClassifier
    ▸ RandomForestClassifier ⓘ

```

```

In [113...] print(f"Лучшие параметры: {grid.best_params}")
            print(f"Лучший ROC-AUC на кросс-валидации: {grid.best_score_:.4f}")

```

Лучшие параметры: {'max_depth': 10, 'min_samples_leaf': 4, 'min_samples_split': 10, 'n_estimators': 300}

Лучший ROC-AUC на кросс-валидации: 0.8573

```

In [115...] best_rf = grid.best_estimator_
            y_pred_best = best_rf.predict(X_test)
            y_proba_best = best_rf.predict_proba(X_test)[: , 1]

```

```

In [117...] print(f"Accuracy: {accuracy_score(y_test, y_pred_best):.4f}")
            print(f"Precision: {precision_score(y_test, y_pred_best):.4f}")
            print(f"Recall: {recall_score(y_test, y_pred_best):.4f}")
            print(f"F1-score: {f1_score(y_test, y_pred_best):.4f}")
            print(f"ROC-AUC: {roc_auc_score(y_test, y_proba_best):.4f}")

```

Accuracy: 0.8084

Precision: 0.6844

Recall: 0.5160

F1-score: 0.5884

ROC-AUC: 0.8529

3.8 Интерпретация

```

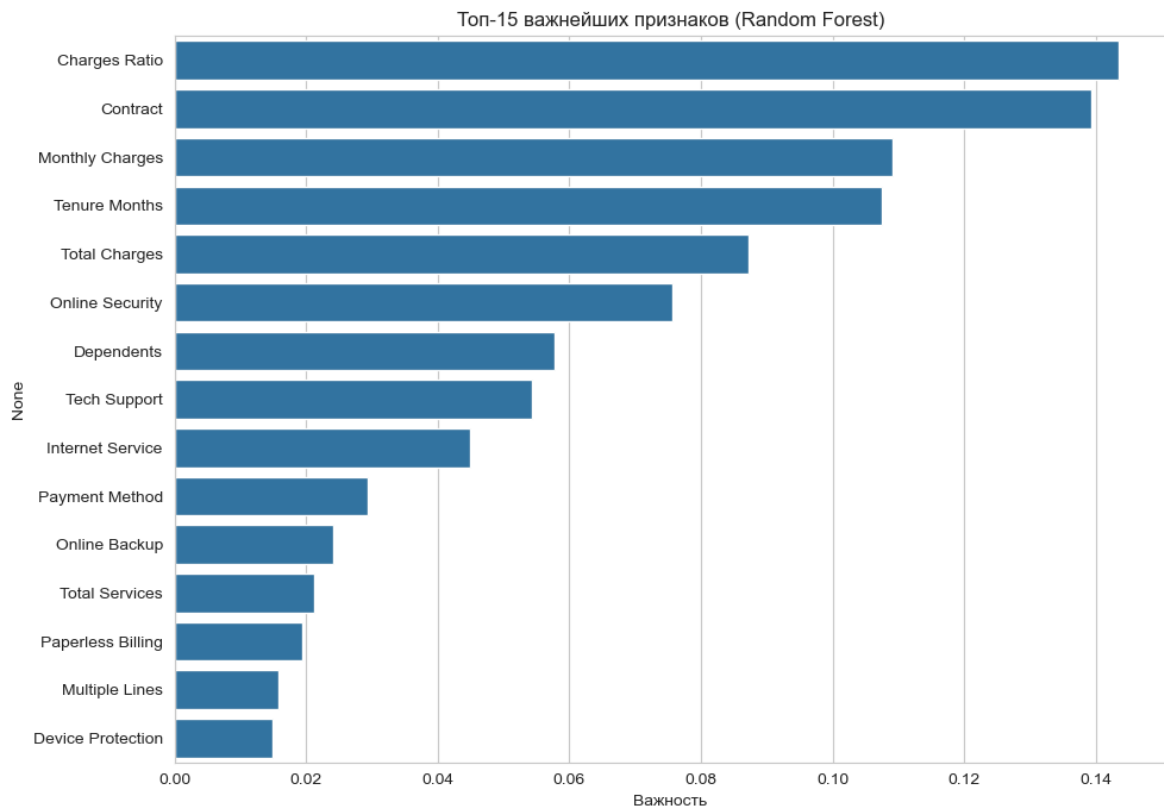
In [120...] importances = best_rf.feature_importances_
            features = X.columns
            feat_imp = pd.Series(importances, index=features).sort_values(ascending=False)

```

```

In [122...] plt.figure(figsize=(10, 7))
            sns.barplot(x=feat_imp.values[:15], y=feat_imp.index[:15])
            plt.title('Топ-15 важнейших признаков (Random Forest)')
            plt.xlabel('Важность')
            plt.tight_layout()
            plt.show()

```



```
In [164... print("Топ-10 признаков:")
print(feat_imp.head(10))
```

Топ-10 признаков:

Charges Ratio	0.143545
Contract	0.139405
Monthly Charges	0.109088
Tenure Months	0.107584
Total Charges	0.087299
Online Security	0.075690
Dependents	0.057820
Tech Support	0.054379
Internet Service	0.044941
Payment Method	0.029333

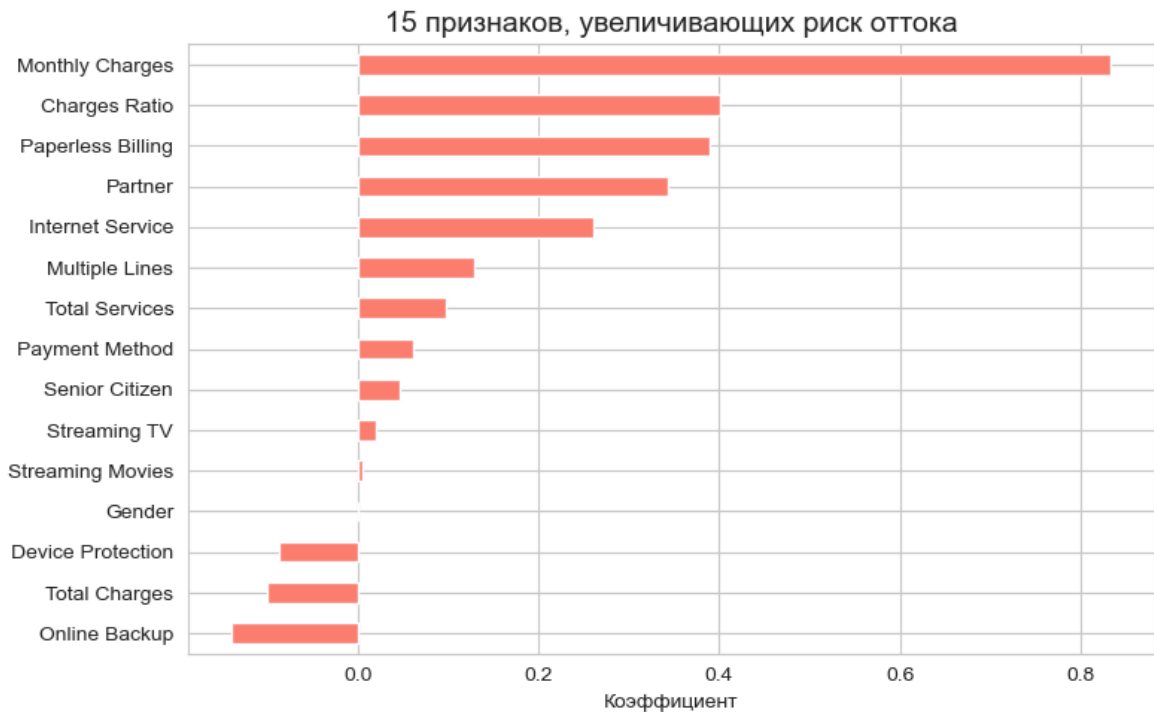
dtype: float64

- **Самым важным признаком оказался созданный нами Charges Ratio.** Это подтверждает гипотезу о том, что отношение текущих трат к общей накопленной сумме (по сути, интенсивность трат клиента) — критический фактор оттока
- Высокая важность признаков Contract и Tenure Months говорит о том, что клиенты на долгосрочных контрактах и те, **кто с компанией давно, уходят значительно реже**
- Monthly Charges входит в топ-3, **чем выше ежемесячный чек, тем выше риск**, что клиент найдет более дешевое предложение у конкурентов

```
In [127... coeffs = pd.Series(lr.coef_[0], index=X.columns).sort_values()
```

```
In [129... plt.figure(figsize=(8, 5))
coeffs.tail(15).plot(kind='barh', color='salmon')
plt.title('15 признаков, увеличивающих риск оттока', fontsize=14)
```

```
plt.xlabel('Коэффициент')
plt.tight_layout()
plt.show()
```



- **Monthly Charges** имеет самый длинный столбец вправо — это значит, чем выше платеж, тем выше риск оттока. (Гипотеза подтверждена).
- **Online Backup и Total Charges** уходят влево (отрицательный коэффициент) — это факторы, которые, наоборот, удерживают клиента.

3.9 Проверка гипотез

Гипотеза 1: Отток зависит от типа контракта

```
In [134... df_raw = db.copy()
```

```
In [136... cross = pd.crosstab(db_raw['Contract'], db_raw['Churn Value'])
chi2, p, dof, ex = stats.chi2_contingency(cross)
print(f'Хи-квадрат: {chi2:.2f}, p-value: {p}')
```

Хи-квадрат: 1184.60, p-value: 5.863038300673391e-258

Вывод: p-value < 0.001 - зависимость статистически значима

Гипотеза 2: Наличие техподдержки снижает отток

```
In [140... cross_ts = pd.crosstab(df_raw['Tech Support'], df_raw['Churn Value'])
chi2_ts, p_ts, _, _ = stats.chi2_contingency(cross_ts)
print(f'Хи-квадрат: {chi2_ts:.2f}, p-value: {p_ts}')
```

Хи-квадрат: 828.20, p-value: 1.4430840279998987e-180

Вывод: p-value ~0.0 - техподдержка значимо связана с оттоком

4. Архитектура

Data → Preprocessing → Model → Evaluation → Сохранение модели

```
In [145... joblib.dump(best_rf, 'churn_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("Модель и скейлер сохранены.")
```

Модель и скейлер сохранены.

Заключение

В ходе работы был реализован полный цикл разработки ML-решения. Внедрение лучшей модели (Gradient Boosting с ROC-AUC 0.85) позволяет бизнесу:

1. Выявлять до 80% потенциально уходящих клиентов
2. Направлять бюджеты (скидки, бонусы) только на те сегменты, где риск оттока подкреплён финансовой значимостью клиента
3. Анализ важности признаков показал, что Charges Ratio и тип контракта — главные драйверы оттока. Рекомендуется пересмотреть стратегию перевода клиентов с «Monthly» на годовые контракты для стабилизации абонентской базы.

Модель готова к интеграции и сохранена в формате .pkl

```
In [149... print('')
```